Daily Quant Digest: 2026-10-21

Today's topics

Track	Topic	Difficulty
Probability & Statistics	Restart, day 3: conditional expectation, the tower property and the law of total variance ("average the group averages")	Medium
Derivatives	The volatility smile and skew: why one option market quotes many vols, on a USDJPY 25-delta put	Medium
Algorithms & DS	Shortest paths: Dijkstra, and counting the number of shortest routes	Medium-Hard
Music	The Who, "Baba O'Riley" (1971)	

1. Probability & Statistics: Conditional Expectation and the Law of Total Variance

1.1 The one idea: split the crowd into groups

Suppose a bank wants the average bonus across its whole trading floor. Nobody adds up every single bonus. The head of each desk reports their desk's average, and HR takes a weighted average of those desk averages, weighted by desk size. Same answer.

That is the **tower property**, the most useful rule in conditional probability:

$$E[X] = E\big[\,E[X \mid G]\,\big].$$

Read it slowly. $E[X \mid G]$ means "the average of X *inside* the group G you are in". It is itself random, because which group you land in is random. Averaging it over the groups gives back the overall average. **The average of the group averages is the overall average.**

There is a second, less obvious idea hiding here. Think of $E[X \mid G]$ as your **best guess** of X when the only thing you know is the group. If someone tells you "this trader is on the rates desk", your best guess of their bonus (in the sense of smallest average squared error) is the rates desk average. You cannot do better with that information. Mathematicians call this a **projection**: you squash X down onto "what can be explained by the group", and the leftover error is pure noise that the group label cannot predict.

1.2 Spread also splits into two pieces

Now the fun part. Why do bonuses vary across the floor? For two reasons:

1. **Within groups:** people on the same desk get different bonuses (noise inside a group).
2. **Between groups:** different desks have different averages (the groups themselves differ).

The **law of total variance** says these two pieces simply add:

$$\operatorname{Var}(X) = \underbrace{E\big[\operatorname{Var}(X \mid G)\big]}_{\text{average spread inside groups}} + \underbrace{\operatorname{Var}\big(E[X \mid G]\big)}_{\text{spread of the group averages}}.$$

A picture: a photo of a crowd standing on a set of stairs. Heights in the photo vary because people differ in height (within) *and* because they stand on different steps (between). Total variation = the person part + the step part.

Students remember it as **"Eve's law"**: $\operatorname{Var} = E[V] + V[E]$.

1.3 A mini example: 1,000 traders

Imagine **1,000 traders**:

Group	Count	Average bonus	Spread (sd) inside group
Junior	600	100k	20k
Senior	400	200k	50k

Overall average (tower property). Total bonus pool is $600 \times 100 + 400 \times 200 = 140{,}000$ k across 1,000 people, so the average is 140k. Equivalently $0.6 \times 100 + 0.4 \times 200 = 140$.

Within part. Each junior's squared distance from the junior average is 400 on average (20^2), each senior's is 2,500 (50^2). Averaged over 1,000 people:

$$0.6 \times 400 + 0.4 \times 2{,}500 = 240 + 1{,}000 = 1{,}240.$$

Between part. Treat every junior as "100" and every senior as "200" and ask how spread out that is around 140:

$$0.6 \times (100 - 140)^2 + 0.4 \times (200 - 140)^2 = 960 + 1{,}440 = 2{,}400.$$

Total. $\text{Var} = 1{,}240 + 2{,}400 = 3{,}640$, so the overall sd is $\sqrt{3{,}640} \approx 60.3$k.

Look at the result: the overall sd (60k) is bigger than the spread inside *either* group (20k and 50k). About two thirds of the variance ($2{,}400/3{,}640 \approx 66\%$) comes only from *not knowing which group someone is in*. Tell me the group and most of the uncertainty disappears.

1.4 Common mistakes

- **Averaging the sds.** The overall sd is not $0.6 \times 20 + 0.4 \times 50 = 32$. You forgot the between part. Variances add, sds do not.
- **Forgetting the weights.** The between part uses the *group probabilities* as weights, not "each group counts equally".
- **Thinking $E[X \mid G]$ is a number.** It is a random variable: a function of the group. That is exactly why it has a variance.

1.5 Interview question

A bag holds a huge number of coins whose probabilities of heads are spread evenly between 0 and 1 (so the bias p of a coin you draw is Uniform(0,1)). You draw one coin, do not look at its bias, and flip it 10 times. Let X be the number of heads.

a. What is $E[X]$?

b. What is $\text{Var}(X)$? Compare with a fair coin.

c. What fraction of that variance comes purely from not knowing p?

d. The 10 flips all come up heads. What is your best guess for p now?

1.6 Solution

Count first. Imagine **1,000 traders**, each drawing their own coin and flipping it 10 times. A trader with a 0.9 coin will mostly see 8 to 10 heads; a trader with a 0.1 coin mostly 0 to 2. So across the room the head counts are very spread out, far more than if everyone had a fair coin. Each trader is a "group" with their own p. That is the law of total variance waiting to be used.

(a) Condition on the coin. Given p, X is Binomial(10, p), so $E[X \mid p] = 10p$. Tower property:

$$E[X] = E[10p] = 10 \times \tfrac{1}{2} = 5.$$

(b) Given p: $\text{Var}(X \mid p) = 10p(1-p)$. Eve's law:

- Within part: $E[10p(1-p)] = 10\big(E[p] - E[p^2]\big) = 10\left(\tfrac{1}{2} - \tfrac{1}{3}\right) = \tfrac{10}{6} \approx 1.67$.
- Between part: $\text{Var}(10p) = 100\,\text{Var}(p) = 100 \times \tfrac{1}{12} \approx 8.33$.

$$\mathrm{Var}(X) = \tfrac{10}{6} + \tfrac{100}{12} = 10.$$

A fair coin gives $10 \times \frac{1}{2} \times \frac{1}{2} = 2.5$. The unknown bias makes the variance **four times bigger**.

(c) Between part / total $= 8.33/10 \approx 83\%$. Most of the uncertainty is about *which coin you have*, not about the flips.

(d) Your best guess is $E[p \mid X = 10]$. With a uniform starting belief, the chance of 10 heads given p is p^{10}, so the updated belief is proportional to p^{10} (Bayes from day 1). Its average is

$$E[p \mid X = 10] = \frac{\int_0^1 p \cdot p^{10}\, dp}{\int_0^1 p^{10}\, dp} = \frac{1/12}{1/11} = \frac{11}{12} \approx 0.917.$$

Interview trick. There is a slick way to see the whole distribution of X. Draw 11 independent uniform numbers: the first is the coin's bias p, the other 10 are the flips (a flip is "heads" if its number is below p). Then X is just *how many of the 10 flip-numbers fall below p*, i.e. the rank of p among 11 uniforms. By symmetry every rank is equally likely, so $P(X = k) = 1/11$ for $k = 0, 1, \ldots, 10$. The general rule $E[p \mid k \text{ heads in } n] = \frac{k+1}{n+2}$ is Laplace's "rule of succession".

Sanity check. A uniform distribution on $\{0, \ldots, 10\}$ has mean 5 and variance $\frac{11^2 - 1}{12} = 10$. Both match (a) and (b) exactly. A simulation of 2 million draws gives mean 4.999, variance 9.998, each count near 0.091, and an average bias of 0.917 among all-heads runs.

---

2. Derivatives: The Volatility Smile and Skew

2.1 The setup: who trades this and why

A Japanese car exporter sells cars in the US and will receive USD 10 million in one month. Spot USDJPY is 150.00. If the yen strengthens (USDJPY falls), those dollars buy fewer yen and the profit shrinks. So the treasurer buys a **USD put / JPY call**: the right to sell USD 10M at, say, 146.50 in one month.

On the other side sits a bank's FX options desk. It sells the option, collects the premium, and hedges. Hedge funds running carry trades (borrow yen cheaply, buy higher-yielding dollars) are also big buyers of these puts: a yen spike is exactly the disaster that wipes them out.

The treasurer asks for a price. The dealer says "**11.3 vol**". But the at-the-money option in the same expiry is quoted at **10.0 vol**. Same currency pair, same date, different volatility. If Black-Scholes were literally true, there would be only one volatility. So what is going on?

2.2 Implied vol is just a price unit

First, a mindset shift. **Implied vol is not a forecast. It is a way of quoting a price.**

Analogy: a mortgage is quoted as an interest rate, not as "you pay 2.1 million in total". The rate is easier to compare across loans of different sizes and lengths. In the same way, an option's dollar price is hard to compare across strikes and expiries, so traders put the price *into* the Black-Scholes formula and ask "what volatility makes the formula give this price?" That number is the implied vol.

So when the dealer says 11.3 for the 146.50 put and 10.0 for the at-the-money option, he is saying: "this put is **more expensive** than the simple formula with 10% would suggest." Plot implied vol against strike and you get a curve, not a flat line. If it curves up on both sides it is a **smile**. If one side is much higher than the other it is a **skew**.

2.3 Why the curve is not flat: fat tails

Black-Scholes assumes returns follow a bell curve with a fixed width. Think of it as a weather model that knows about drizzle and sunshine but has never heard of typhoons.

Let us test it. With 10% annual vol, one month's sd is $10\%/\sqrt{12} \approx 2.9\%$. A 10% fall in USDJPY within a month is then about a 3.6-sigma event. The bell curve says that happens with probability about 0.013% per month: **once every 600 years or so**.

Reality: in July to August 2024, USDJPY fell from about 162 to about 142 in roughly a month as carry trades unwound, a move of more than 12%. In October 1998, around the LTCM crisis, it fell more than 10% in a few days. Big jumps happen far more often than the bell curve allows. Real return distributions have **fat tails**.

Options far from the money are exactly bets on those tails. If the formula underestimates how often tails occur, the market corrects it the only way it can: by plugging a **higher vol** into the formula for those strikes. The smile is the market patching the model's blind spot.

2.4 Why one side is higher: skew

Fat tails explain why *both* wings are lifted. Skew explains why one wing is lifted *more*. Two forces:

1. **Which way the crashes go.** In equities, markets fall fast and rise slowly ("up the stairs, down the elevator"). And when stocks fall, volatility rises. So low strikes (puts) carry higher vol. This **equity skew** has been permanent since the October 1987 crash.
2. **Who needs insurance.** Pension funds buy index puts to protect portfolios; exporters and carry traders buy yen calls. Persistent demand on one side pushes that side's price, and its vol, up. Like flood insurance in a river town: everyone wants it, so it is never cheap.

FX is interesting because there is no “natural” down direction: a USD put is a JPY call. The skew points toward whichever currency tends to **jump in a panic**. The yen is a safe haven: in risk-off episodes carry trades unwind and the yen surges. So USDJPY options have a skew toward **low strikes** (USD puts / JPY calls). EURUSD, by contrast, is often close to a symmetric smile.

FX traders summarise the shape with two numbers (topic 14 will go deeper):

- **25-delta risk reversal (RR):** vol of the 25-delta call minus vol of the 25-delta put. It measures the **tilt**. Negative for USDJPY means USD puts are richer.
- **25-delta butterfly (BF):** roughly, the average of the two wings minus ATM. It measures the **curvature**, how fat the tails are.

2.5 The worked trade

Market (illustrative): spot 150.00, 1 month, USD rate 4.0%, JPY rate 0.75%. The forward is about 149.59 (lower than spot because the yen earns less interest). ATM vol 10.0%, 25d RR $= -2.0$, 25d BF $= 0.3$.

So, approximately:

- 25d put vol $= 10.0 + 0.3 + 1.0 = 11.3\%$
- 25d call vol $= 10.0 + 0.3 - 1.0 = 9.3\%$

At 11.3% vol, the 25-delta USD put has a strike of about 146.43; round it to **146.50**. Price it with Garman-Kohlhagen (Black-Scholes for FX), notional USD 10M:

Vol used	Premium (JPY per USD)	Premium on USD 10M	% of notional	Delta
Flat 10.0% (wrong)	0.586	JPY 5.86M (about USD 39,000)	0.39%	-0.23
Smile 11.3% (market)	0.761	JPY 7.61M (about USD 50,800)	0.51%	-0.25

The smile makes the put about **30% more expensive** (JPY 1.76M, roughly USD 11,700). Quick check with vega: this option’s vega is about JPY 0.13 per USD per vol point, and $0.13 \times 1.3 \approx 0.17$, close to the true difference of 0.176.

What it means for each side:

- **The exporter** pays more for protection because everyone else wants the same protection. If he only needs to cover a gradual drift, he could sell the cheaper 25d USD call (9.3 vol) to fund it: that is a **risk reversal** trade, a “collar”.
- **The dealer** who sells the put at 11.3 and hedges it as if vol were 10 is fine in quiet markets (he earns extra premium), but loses badly in a jump, since delta hedging cannot follow a gap. The extra 1.3 vols are his payment for that jump risk.

2.6 When it goes wrong

In January 2015 the Swiss National Bank suddenly removed the 1.20 floor on EURCHF. The franc jumped about 30% within minutes, then settled near 15 to 20% stronger. Dealers and brokers who had sold franc calls cheaply, assuming the floor would hold, took huge losses; several retail FX brokers failed. The market had priced almost no skew for a move the central bank was holding back by force. Lesson: a smile that looks "too flat" may be the most dangerous one.

2.7 Likely interview questions

Q1. Why is equity index skew downward sloping? Crashes are downward and fast, volatility rises when prices fall (spot-vol correlation is negative), and investors persistently pay up for downside puts as portfolio insurance.

Q2. USDJPY 1M 25d RR is -2.0. What does it tell you? USD puts (JPY calls) are 2 vols more expensive than equally out-of-the-money USD calls. The market fears a sharp yen rally (risk-off, carry unwind) more than a sharp yen sell-off.

Q3. What does the smile tell you about the market's distribution? The second derivative of call prices with respect to strike gives the risk-neutral density: $\frac{\partial^2 C}{\partial K^2} = e^{-rT} f(K)$ (Breeden-Litzenberger). In practice: a tight butterfly's price is proportional to the probability of ending near its middle strike. Higher wings mean fatter tails than a bell curve.

---

3. Algorithms & Data Structures: Number of Shortest Routes (Dijkstra)

3.1 Problem

A city has n intersections numbered 0 to $n-1$ and m two-way roads. Road i connects u_i and v_i and takes t_i minutes. You start at intersection 0 and want to reach $n-1$ as fast as possible.

Return the **number of different routes** that achieve the minimum travel time, modulo 10^9+7.

Constraints

- $2 \le n \le 2 \times 10^5$, $n-1 \le m \le 2 \times 10^5$
- $1 \le t_i \le 10^9$ (all times are **positive**)
- No duplicate roads, and every intersection is reachable from 0.

Example 1

```
n = 7
roads = [0,6,7] [0,1,2] [1,2,3] [1,3,3] [6,3,3]
        [3,5,1] [6,5,1] [2,5,1] [0,4,5] [4,6,2]
Output: 4
```

The shortest time is 7. The four routes are $0 \to 6$; $0 \to 4 \to 6$; $0 \to 1 \to 2 \to 5 \to 6$; $0 \to 1 \to 3 \to 5 \to 6$.

Example 2

```
n = 2, roads = [1,0,10]
Output: 1
```

This is a classic (LeetCode 1976) and a nice one for interviews: it checks that you know Dijkstra *and* that you understand why it works, because the counting only works if you know exactly when a node's answer is final.

3.2 From first principles

Brute force. Try every route from 0 to $n-1$ (depth-first search over simple paths), track the best time and how many routes tie it. This is correct but the number of paths can be exponential: a chain of 30 "diamonds" (each a choice of two equal roads) already has $2^{30} \approx 10^9$ shortest routes. Hopeless for $n = 2 \times 10^5$.

Step 1: find the shortest time (Dijkstra). Picture pouring water into intersection 0. The water spreads along roads at one metre per minute. Each intersection gets wet at exactly its shortest travel time. Dijkstra's algorithm simulates this flood with a **min-heap** ("which intersection gets wet next?"):

1. $\text{dist}[0] = 0$, everything else ∞. Push $(0, 0)$.
2. Pop the intersection u with the smallest tentative distance. That distance is now **final**: any other route to u would have to pass through some intersection that is still dry, which is already at least as far, and roads only add time (positive weights!).
3. For each road $u \to v$ of length w, if $\text{dist}[u] + w < \text{dist}[v]$, improve $\text{dist}[v]$ and push it.

Step 2: count while you flood. Keep $\text{ways}[v]$ = number of shortest routes to v found so far. When we relax road $u \to v$ with new length $nd = \text{dist}[u] + w$:

- $nd < \text{dist}[v]$: we found a **strictly faster** way. All old routes are now useless. Set $\text{ways}[v] = \text{ways}[u]$.
- $nd = \text{dist}[v]$: a **tie**. Every shortest route to u, extended by this road, is another shortest route to v. Add: $\text{ways}[v] \mathrel{+}= \text{ways}[u]$.
- $nd > \text{dist}[v]$: ignore.

Why is $\text{ways}[u]$ complete when we use it? Any intersection that sits just before u on a shortest route has a strictly smaller distance (positive weights), so it was popped, and passed on its count, *before* u was popped. So by the time u pushes its count forward, it has

received every contribution. This is the key insight, and it is also why the trick breaks with zero-weight roads: two intersections at the same distance could feed each other in the wrong order.

Stale heap entries. A node can be pushed several times as its distance improves. When an old, larger entry is popped later, we skip it (`if (d > dist[u]) continue;`). This "lazy deletion" is simpler than a decrease-key operation, which `std::priority_queue` does not support.

Complexity. Each road is relaxed at most twice (once from each end), and each push and pop costs $O(\log m)$. Time $O((n + m) \log m)$, space $O(n + m)$.

Overflow. A shortest route can have up to 2×10^5 roads of 10^9 minutes: about 2×10^{14}, far beyond `int`. Distances must be `long long`. Counts are reduced mod $10^9 + 7$ at each addition.

When Dijkstra fails: Bellman-Ford. With negative edges the "once popped, final" argument dies. Example from FX: converting through rates r_{ij}, the best conversion maximises a product of rates, which is the same as minimising $\sum -\log r_{ij}$. Those weights can be negative, and a negative cycle is an **arbitrage loop**. There you use Bellman-Ford: relax every edge $n - 1$ times ($O(nm)$); if a further round still improves something, a negative cycle exists. We will do that in full on a later day.

3.3 C++ solution

```cpp
#include <array>
#include <functional>
#include <limits>
#include <queue>
#include <utility>
#include <vector>

using std::vector;
using ll = long long;

constexpr ll MOD = 1'000'000'007LL;
constexpr ll INF = std::numeric_limits<ll>::max();

int countPaths(int n, const vector<std::array<int, 3>>& roads) {
    // Adjacency list: adj[u] holds (neighbour, travel time) pairs.
    vector<vector<std::pair<int, int>>> adj(n);
    for (const auto& r : roads) {
        adj[r[0]].push_back({r[1], r[2]});
        adj[r[1]].push_back({r[0], r[2]});   // roads are two-way
    }

    vector<ll> dist(n, INF);
    vector<ll> ways(n, 0);
    dist[0] = 0;
    ways[0] = 1;

    // Min-heap of (distance, node). std::greater flips the default max-heap.
    using Item = std::pair<ll, int>;
    std::priority_queue<Item, vector<Item>, std::greater<Item>> pq;
    pq.push({0, 0});
```

```
    while (!pq.empty()) {
        auto [d, u] = pq.top();   // structured binding (C++17)
        pq.pop();
        if (d > dist[u]) continue;  // stale entry: u was already settled cheaper

        for (const auto& [v, w] : adj[u]) {
            ll nd = d + w;
            if (nd < dist[v]) {            // strictly better route
                dist[v] = nd;
                ways[v] = ways[u];
                pq.push({nd, v});
            } else if (nd == dist[v]) {    // a tie: one more family of routes
                ways[v] = (ways[v] + ways[u]) % MOD;
            }
        }
    }
    return static_cast<int>(ways[n - 1]);
}
```

The full file `2026-10-21.cpp` adds a brute-force DFS over all simple paths and a test harness: both examples, the 30-diamond ladder (answer $2^{30} \bmod (10^9 + 7) = 73{,}741{,}817$), and a randomized cross-check on 3,000 small graphs with weights 1 to 3 (lots of ties, the tricky case). All pass.

3.4 Walkthrough of the C++

- **`using ll = long long;`** A type alias, the modern form of `typedef`. It keeps the distance type short and easy to change.
- **`constexpr ll MOD = 1'000'000'007LL;`** `constexpr` means the value is known at compile time. The apostrophes are C++14 digit separators, purely for readability. `LL` makes the literal a `long long`.
- **`std::numeric_limits<ll>::max()`** from `<limits>` gives the largest `long long`, used as "infinity". It is safe here because we only add to `d` after popping a real distance, never to `INF` itself.
- **`vector<std::array<int, 3>>`** Each road is a fixed-size array of three ints. Each `std::array` is stored inline in the vector's single buffer, with no extra heap allocation per road, unlike `vector<vector<int>>`.
- **`const vector<...>& roads`** Pass by **const reference**: no copy of the input, and the compiler stops us from modifying it by accident.
- **`vector<vector<std::pair<int,int>>> adj(n);`** The adjacency list: `adj[u]` is a list of `(neighbour, weight)` pairs. Constructing with `(n)` creates `n` empty inner vectors.
- **`for (const auto& r : roads)`** A range-based for loop. `auto` deduces the element type, `const auto&` avoids copying each road.
- **`push_back({r[1], r[2]})`** The braces build a `std::pair<int,int>` in place from two values (brace initialisation).
- **`vector<ll> dist(n, INF);`** The two-argument constructor fills all `n` slots with `INF`.
- **The heap type.** `std::priority_queue<T, Container, Compare>` is a **max-heap** by default (largest on top). Passing `std::greater<Item>` (from `<functional>`) reverses the comparison so the *smallest* pair is on top. Pairs compare lexicographically: first by distance, then by node id, which is exactly what we want.

- `auto [d, u] = pq.top();` A C++17 **structured binding**: unpacks the pair into two named variables. We copy (not reference) because `pop()` destroys the top element right after; a reference would dangle.
- `for (const auto& [v, w] : adj[u])` Structured bindings also work inside range-for, giving readable names to each pair's fields.
- `ll nd = d + w;` `w` is an `int`, `d` is `long long`; the `int` is promoted, so the sum is computed in 64 bits with no overflow.
- `ways[v] = (ways[v] + ways[u]) % MOD;` Both values are below `MOD`, so the sum is below 2.1×10^9, which fits in `long long` easily. (It would just squeeze into a 32-bit `int` too, but one more addition before the `%` would not, so `ll` is the safe habit.)
- `static_cast<int>(...)` An explicit, checked-at-compile-time conversion. Prefer it over C-style `(int)x` because it is easy to search for and cannot silently cast away `const`.
- **In the test file:** the brute force uses `std::function<void(int, ll)> dfs = [&](int u, ll t) {...};`, a **lambda** that captures everything by reference (`[&]`). It must be stored in a `std::function` (not `auto`) so it can call itself recursively. Random graphs come from `std::mt19937`, the Mersenne Twister generator, seeded with a fixed number so failures are reproducible.

---

4. Music: The Who, "Baba O'Riley" (1971)

Context

"Baba O'Riley" opens **Who's Next**, released in August 1971 and widely seen as The Who's finest studio album. It was written by guitarist Pete Townshend. Roger Daltrey sings lead, while Townshend takes the softer middle section.

The story behind it

After the success of the rock opera *Tommy* (1969), Townshend planned something even bigger: **Lifehouse**, a science-fiction project about a future where people live sealed inside suits and music becomes a way back to real life. The concept was too ambitious, the band and management could not make sense of it, and it was abandoned. The best songs were rescued and became *Who's Next*. "Baba O'Riley" was one of them.

The odd title honours Townshend's two big influences at the time: **Meher Baba**, the Indian spiritual teacher he followed, and **Terry Riley**, the American minimalist composer whose music is built from short repeating patterns. The song is often wrongly called by a phrase from its chorus. Townshend has said the lyrics were inspired by the scene at festivals like Woodstock: crowds of young fans left wasted and lost in the mud.

Why people love it

It sounds like nothing else from 1971. The famous opening is a **Lowrey organ** played through a setting that repeats each note in a rapid, bubbling pulse, a direct nod to Riley's minimalism. It feels electronic and hypnotic, decades before that was common in rock. Then Keith Moon's drums crash in, and the song turns into a full rock anthem. It has become a stadium favourite and was used as the theme of the TV series *CSI: NY*.

What to listen for

- **The first minute.** Just the pulsing organ pattern, with a piano joining in. Notice how it never changes, yet feels like it is building.
- **The drum entrance.** Moon does not just keep time; he plays around and through the beat. Listen for his fills.
- **The vocal handover.** Daltrey's powerful voice for the verses, then Townshend's gentler voice in the bridge: a deliberate contrast.
- **The ending.** The tempo speeds up into a frantic, folk-like jig led by a violin, played by guest musician Dave Arbus of the band East of Eden. It ends in a joyous, chaotic rush, the opposite of the calm machine-like opening.

Play it loud.